

Conducción Autónoma en Duckietown mediante Aprendizaje por Refuerzo profundo

Métodos Avanzados en Machine Learning

Adolfo Peña Marín

Junio 2026

1 Introducción y objetivo

El objetivo es entrenar un agente capaz de mantenerse en el carril en Duckietown a partir de observaciones de imagen y que **generalice** a un mapa no visto durante el entrenamiento. La nota depende del desempeño en el mapa oculto `Duckietown-loop_obstacles-v0`, con obstáculos estáticos.

La entrega final es RL puro. Siguiendo las diapositivas del reto, la solución es aprendizaje por refuerzo *de extremo a extremo*: la red recibe la imagen y produce directamente la acción de control (velocidades de rueda). **No** se entrega ningún controlador clásico, PID, heurística de avance forzado (`forward_turn`) ni *action mode* diseñado para conducir (maniobras discretas predefinidas o velocidades acotadas a mano). Esas variantes se exploraron como diagnóstico y se descartan por inyectar conocimiento de conducción.

Contrato de evaluación. El modelo debe (i) esperar observaciones de forma `(1, 64, 64)` apiladas en 4 frames $\rightarrow (4, 64, 64)$; (ii) llevar definidas e importables las clases personalizadas (`CustomCNN`, `wrappers`); y (iii) cargar “a la primera” en un entorno limpio reconstruido desde `requirements.txt`. Entrenar en el mapa oculto supone descalificación: está bloqueado por `ValueError` en tres puntos del código.

Fase 1 (calentamiento): Q-Learning tabular. Como ejercicio previo se implementa Q-Learning tabular *sin librerías de Deep RL* sobre `FrozenLake-v1` 8×8 resbaladiza (actualización de Bellman TD(0), ϵ -greedy con decaimiento, $\alpha = 0.1$, $\gamma = 0.99$, 50 000 episodios). La política greedy alcanza una **tasa de éxito** ≈ 0.52 (Figura 1); código en `q_learning_sandbox.py`. El resto de la memoria se centra en Duckietown.

2 Entorno y pipeline

Duckietown real (gym-duckietown). El simulador usa la API de *gym clásico* (0.25.2), no Gymnasium, por compatibilidad con `gym-duckietown` (daffy, 6.1.34). `DuckieWrapper` (`src/wrappers.py`) adapta esa API antigua (4-tupla) a Gymnasium (5-tupla) que espera Stable-Baselines3. El entorno real se confirma por consola: `[duckie_factory] Using real Duckietown env: ..., gym-duckietown version 6.1.34, using DuckietownEnv.`

Observaciones. Cada frame se procesa así:

RGB $(480 \times 640 \times 3) \rightarrow$ recorte 50% superior (cielo) $\rightarrow$ grises $\rightarrow$ resize $64 \times 64 \rightarrow (1, 64, 64)$ `uint8`,

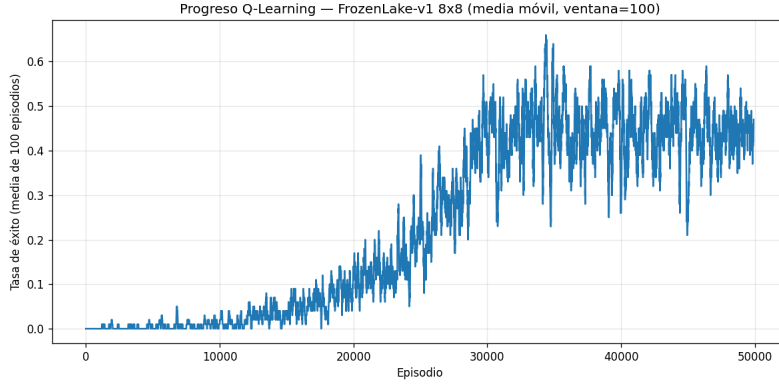


Figure 1: Fase 1: media móvil (ventana 100) de la recompensa por episodio en **FrozenLake-v1** 8×8 resbaladiza; se estabiliza en torno a 0.45–0.50.

y tras `VecFrameStack(n_stack=4)` (`src/envs.py`) $\rightarrow (4, 64, 64)$. El apilado de 4 frames da percepción de velocidad y giro.

Acción continua de ruedas. La acción del simulador son las velocidades de las **dos ruedas** $[v_{izq}, v_{der}] \in [-1, 1]^2$ (`action_mode=wheels`); la política las produce directamente, sin acotar.

Arquitectura y algoritmos. `CustomCNN` (`src/cnn.py`) es un extractor estilo *NatureCNN* (3 conv $\rightarrow$ lineal a 256), común a los tres algoritmos. SB3 ya normaliza la imagen `uint8` a $[0, 1]$ antes del extractor, por lo que la CNN **no** vuelve a dividir entre 255 (Sección 4). Se evalúan:

- **PPO** (on-policy, continuo): $lr = 3 \cdot 10^{-4}$, `n_steps`= 2048, `batch`= 64, `n_epochs`= 10, $\gamma = 0.99$, $\lambda_{GAE} = 0.95$, `clip`= 0.2. Base del modelo final.
- **DQN** (off-policy, discreto vía `DiscreteActionWrapper`): $lr = 10^{-4}$, `buffer`= 50k, `learning_starts`= 10k, `batch`= 32.
- **SAC** (off-policy, máxima entropía, continuo): $lr = 3 \cdot 10^{-4}$, `buffer`= 50k, `batch`= 256, $\tau = 0.005$, `ent_coef`=auto.

Infraestructura. El entrenamiento se realizó en Google Colab con `venv` Python 3.11 (subproceso), `xvfb-run + MPLBACKEND=Agg`, y el orden ***model-first*** (`--init-order model-first`) para evitar un *segmentation fault* de SB3 al inicializar con el entorno real (construir el modelo sobre un entorno sintético y luego `set_env(real)`).

3 Proceso experimental

El desarrollo fue iterativo. En orden cronológico:

1. **Baseline antiguo.** Un PPO previo que solo conducía con una transformación *swap-negate* de la acción (`action_mode=wheels_fixed`); -869.9 en `loop_obstacles`. Fallback inicial.
2. **PPO / DQN / SAC 20k.** Baselines de RL puro a 20k pasos; ninguno superó al baseline antiguo (-924 , -1005 , -1022 respectivamente).
3. **PPO 50k (single-map).** Más pasos sin corregir nada: *peor* (-1481.8).
4. **PPO multimap 50k.** Entrenado en los 5 mapas permitidos; tampoco ayudó (-1764.9 , el peor).
5. **Auditoría técnica.** Revisión de código que reveló dos fallos críticos: la **doble normalización** de la CNN y el vocabulario discreto incoherente de DQN (Sección 4).

6. **Corrección de la doble normalización.** `src/cnn.py`: se elimina la división extra entre 255. Cambio decisivo.
 7. **PPO CNN-fix 50k.** Con la CNN corregida, el mismo PPO empieza a producir episodios completos con recompensa positiva (mejor *rollout* 1225.7), aunque su media sigue siendo negativa y muy dispersa.
 8. **Fine-tuning PPO CNN-fix 100k.** Partiendo del 50k se afinan 50k pasos adicionales (`--init-model`, `lr=5 · 10-5`, RL puro, solo `loop_empty`). Es el modelo final.
- `loop_obstacles` se usó **solo para evaluación**, nunca para entrenar.

4 Análisis de fallos

El hallazgo principal del proyecto es que varios resultados negativos no medían la calidad del agente, sino fallos del *pipeline*:

- **Mock silencioso.** La fábrica de entornos caía *en silencio* a un `MockDuckietownEnv` de **ruido aleatorio** si `gym_duckietown` no se registraba. *Fix*: `duckie_factory.py` falla explícito con `use_mock=False` (nunca cae al mock) e **importa `gym_duckietown` antes de `gym.make`** (ese import registra los IDs).
- **Render / vídeo.** `render()` daba framebuffer ruidoso bajo Xvfb. *Fix*: usar la observación RGB cruda que guarda `DuckieWrapper` en `last_rgb_frame` (`--video-source wrapper_rgb`); solo afecta a la visualización.
- **Convención de acciones del baseline.** El baseline antiguo solo conducía con `wheels_fixed` (su política quedó acoplada a una convención de ruedas). Para un agente entrenado desde cero la convención es indiferente; el modelo final usa `wheels` y **no** requiere truco alguno en evaluación.
- **Doble normalización de la CNN (crítico).** `CustomCNN.forward` dividía la entrada entre 255, pero SB3 *ya* normaliza las imágenes uint8 a `[0,1]` antes del extractor. La CNN recibía valores ~ 255 veces más pequeños. Verificado en Colab:

```
preprocess_obs max: 0.7843137... # SB3 ya normaliza a [0,1]
after extra /255 max: 0.0030757... # la CNN recibia ~255x menos senal
```

Fix: `x = observations.float()` (sin /255). Afectaba a *todos* los algoritmos.

- **DQN con acciones discretas incoherentes.** `DISCRETE_ACTIONS` se diseñó con semántica `[v, δ]`, pero en `action_mode=wheels` se interpreta como `[vizq, vder]`: la acción “recto” `[0.6, 0.0]` se ejecuta como giro brusco. DQN **carece de un avance recto real**; queda como baseline **sospechoso/descartado**.
- **Presupuesto de entrenamiento limitado.** 20k–100k pasos desde píxeles es poco para Duckietown (suele requerir órdenes de magnitud más); contribuye a las medias negativas y a la varianza.

5 Resultados comparativos

Todas las cifras usan la infraestructura corregida (entorno real con fallo explícito, `model-first`); las cifras positivas reportadas antes de estos arreglos no se reproducían y quedan superadas. Métrica principal: recompensa en el mapa oculto `loop_obstacles`. La Tabla 1 recoge las **medias de evaluación** (`eval.py`) y la Tabla 2 separa los *best-of-10* cualitativos en vídeo.

Modelo	reward medio	std	long. media	observaciones
Baseline antiguo (wheels_fixed)	−869.9	548.6	1264.4	fallback inicial
PPO 20k (<i>bug CNN</i>)	−924.2	441.4	—	no supera baseline
DQN 20k (<i>bug CNN</i>)	−1004.6	30.0	—	descartado (vocab. acción)
SAC 20k (<i>bug CNN</i>)	−1021.7	81.4	—	no supera baseline
PPO 50k single-map (<i>pre-fix</i>)	−1481.8	473.3	455.6	más pasos no ayudan
PPO multimap 50k (<i>pre-fix</i>)	−1764.9	712.8	460.0	multimapa no ayuda
PPO CNN-fix 50k	−2038.0	1408.0	1172.0	rollouts positivos, alta varianza
PPO CNN-fix 100k (ft)	−487.6	1049.9	742.2	modelo final

Table 1: Medias de evaluación en `loop_obstacles` (solo evaluación, nunca entrenamiento), `eval.py`, política determinista. “CNN-fix” = doble normalización corregida. El modelo final mejora la media del baseline antiguo (−487.6 vs −869.9). Para referencia, su evaluación en `loop_empty`: $-1098.3 \pm 742.7 / 501.8$.

Modelo	mejor rollout	rollouts positivos completos	notas
PPO multimap 50k (<i>pre-fix</i>)	−887.8	0 / 1	rollout aislado, 257 frames
PPO CNN-fix 50k	1225.7	2 / 10	alta varianza (−3445 a +1226)
PPO CNN-fix 100k (ft)	1249.5	6 / 10	mejor rollout, 1501 frames

Table 2: *Best-of-10* cualitativo en `loop_obstacles` (`make_eval_video.py`, `--video-source wrapper_rgb`). “rollouts positivos completos” = episodios que llegan al límite de pasos con recompensa acumulada positiva. El modelo final pasa de 2/10 (a 50k) a **6/10** (a 100k), con el mejor *rollout* en 1249.5.

Lectura. Antes del *fix*, todos los modelos rondaban −900 a −1800 de media y ningún *rollout* era positivo. Con la CNN corregida aparecen *rollouts* completos positivos; el fine-tuning a 100k sube la media de −2038 a −487.6 y los *rollouts* positivos de 2/10 a 6/10. La media sigue negativa por la **alta varianza**: el agente conduce bien a veces y se sale otras según la pose inicial.

6 Modelo final

El modelo entregado es `best_agent.zip`, copia de `ppo_cnnfix_loop_empty_100k_ft.zip`: un **PPO de RL puro** (50k + fine-tuning a 100k) entrenado en `loop_empty` con `action_mode=wheels` y la CNN con la normalización corregida.

Justificación. Es el mejor de la familia corregida y **mejora frente al baseline antiguo**: recompensa media −487.6 en `loop_obstacles` (vs −869.9) y **6 de 10** episodios completos con recompensa positiva. El **vídeo final** (best-of-10, cámara real `wrapper_rgb`) muestra el mejor *rollout* con recompensa 1249.527 y 1501 **frames** (vuelta completa sin salirse), en `delivery/final_agent_loop_obstacles.mp4`. Además, a diferencia del baseline antiguo, **carga y se evalúa sin ningún `--action-mode`** (usa `wheels` por defecto), lo que lo hace directamente compatible con el contrato del profesor.

Pureza del enfoque. El modelo final no usa `safe_discrete` (maniobras predefinidas), `v_omega_safe` (velocidad/giro acotados a mano), `forward_turn` (no implementado), PID ni controladores: aprende de la imagen a velocidades de rueda directas. Esas variantes se exploraron y se descartan por inyectar conocimiento de conducción.

Dónde reproducirlo. El stack exacto y el protocolo completo de entrenamiento/evaluación están en `requirements.txt`, `COLAB_SETUP.md` y `EXPERIMENTS.md`; el modelo final se entrena con `action_mode=wheels` y se evalúa *sin* `--action-mode`.

7 Limitaciones y trabajo futuro

- **Alta varianza entre *resets*:** el desempeño depende mucho de la pose inicial (*rollouts* de +1249 a −1397). Es la limitación principal.
- **Más pasos de entrenamiento:** 50k–100k es poco; entrenar $\geq 300\text{k}$ –1M pasos debería reducir la varianza y subir la media.
- **Ajuste de hiperparámetros:** p.ej. `ent_coef` > 0 para más exploración, o *schedules* de `lr/clip`.
- **Revisión de DQN:** alinear el vocabulario discreto con la semántica de ruedas (incluir un avance recto real) para un baseline discreto honesto.
- **Vectorización real (SubprocVecEnv):** varios entornos en paralelo para hacer viable en tiempo el presupuesto alto; y envolver la evaluación con `Monitor` para métricas de episodio más limpias.